



II-35 Estructuras de Datos y Algoritmos

“INVESTIGACION”

Análisis de Algoritmos de Ordenamiento Básicos

Estudiante

Carlos Mauricio Delgado
Rodriguez

Profesor:

Fabián Chinchilla Mayorga

Año

2026

Índice:

Aplicaciones prácticas del ordenamiento de datos en el mundo real	3
1. Sistemas de gestión de estudiantes o empleados	3
2. Comercio electrónico y plataformas de ventas en línea.....	3
2. Algoritmos de ordenamiento.....	4
2.1 Ordenamiento de Burbuja (Bubble Sort).....	4
2.2 Ordenamiento por Selección (Selection Sort)	6
2.3 Ordenamiento por Hundimiento (Sink Sort)	8
2.4 Ordenamiento Rápido (Quick Sort)	10
4. Comparación entre los algoritmos de ordenamiento	12
4.1 Tabla comparativa de los algoritmos.....	12
4.2 Diferencias clave en términos de eficiencia y facilidad de implementación.....	12
4.3 Escenarios donde cada algoritmo podría ser preferible	13
Bubble Sort.....	13
Selection Sort.....	13
Sink Sort	13
Quick Sort.....	13
5. Reflexión crítica.....	13
5.1 Limitación común de los algoritmos estudiados	13
5.2 Relevancia de estos algoritmos en la programación moderna	14
6. Conclusiones.....	14
Referencias	15

1. Introducción a los algoritmos de ordenamiento

¿Qué es un algoritmo de ordenamiento y cuál es su importancia en programación?

Un **algoritmo de ordenamiento** es un conjunto de instrucciones diseñadas para organizar un conjunto de datos siguiendo un criterio específico, como el orden ascendente o descendente. Estos algoritmos permiten reorganizar los elementos de una estructura de datos, como arreglos o listas, de manera que la información sea más fácil de consultar, analizar y procesar. El objetivo principal es mejorar la eficiencia en las operaciones de búsqueda, almacenamiento y manipulación de la información.

Los algoritmos de ordenamiento son fundamentales en la programación porque muchas aplicaciones requieren trabajar con grandes volúmenes de datos. Cuando la información se encuentra ordenada, otros algoritmos, como la búsqueda binaria, pueden ejecutarse con mayor rapidez y eficiencia. Además, el ordenamiento facilita el análisis de datos, la generación de reportes y la organización de registros en sistemas informáticos. Por esta razón, constituyen una de las bases del estudio de los algoritmos y las estructuras de datos (Fritelli et al., 2020; Zohonero Martínez & Joyanes Aguilar, 2008).

Según Ayala San Martín (2020), seleccionar el algoritmo de ordenamiento adecuado depende de factores como el tamaño del conjunto de datos, la cantidad de memoria disponible y el tiempo de ejecución requerido, ya que cada algoritmo presenta ventajas y limitaciones en función del problema que se desea resolver.

Aplicaciones prácticas del ordenamiento de datos en el mundo real

Los algoritmos de ordenamiento tienen múltiples aplicaciones en diferentes áreas de la informática. Dos ejemplos son los siguientes:

1. Sistemas de gestión de estudiantes o empleados

En instituciones educativas y empresas es común ordenar la información de estudiantes o empleados por nombre, número de identificación, promedio o fecha de ingreso. Esto permite localizar registros con mayor rapidez, generar reportes organizados y mejorar la administración de la información.

2. Comercio electrónico y plataformas de ventas en línea

Las tiendas virtuales utilizan algoritmos de ordenamiento para presentar los productos según diferentes criterios, como precio, popularidad, calificación de los usuarios o fecha de publicación.

De esta forma, los clientes pueden encontrar más fácilmente los productos que mejor se ajustan a sus necesidades, mejorando la experiencia de compra.

2. Algoritmos de ordenamiento

2.1 Ordenamiento de Burbuja (Bubble Sort)

Descripción

El **ordenamiento de burbuja (Bubble Sort)** es uno de los algoritmos de ordenamiento más sencillos y utilizados para introducir el estudio de los algoritmos. Su funcionamiento consiste en comparar pares de elementos adyacentes dentro de un arreglo e intercambiarlos cuando se encuentran en un orden incorrecto. Este proceso se repite varias veces hasta que no sea necesario realizar más intercambios, lo que indica que la lista se encuentra completamente ordenada.

Su nombre proviene del comportamiento de los elementos durante el proceso de ordenamiento, ya que los valores mayores "suben" gradualmente hacia el final del arreglo, de forma similar a las burbujas que ascienden en un líquido (Fritelli et al., 2020).

Funcionamiento paso a paso

Supóngase el siguiente arreglo:

8, 5, 3, 9, 1

Primera pasada

- Se compara 8 con 5 → se intercambian.
- Se compara 8 con 3 → se intercambian.
- Se compara 8 con 9 → no se intercambian.
- Se compara 9 con 1 → se intercambian.

Resultado:

5, 3, 8, 1, 9

El número **9** ya quedó en su posición definitiva.

Segunda pasada

- 5 y 3 → intercambio.
- 5 y 8 → correcto.
- 8 y 1 → intercambio.

Resultado:

3, 5, 1, 8, 9

Tercera pasada

- 3 y 5 → correcto.
- 5 y 1 → intercambio.

Resultado:

3, 1, 5, 8, 9

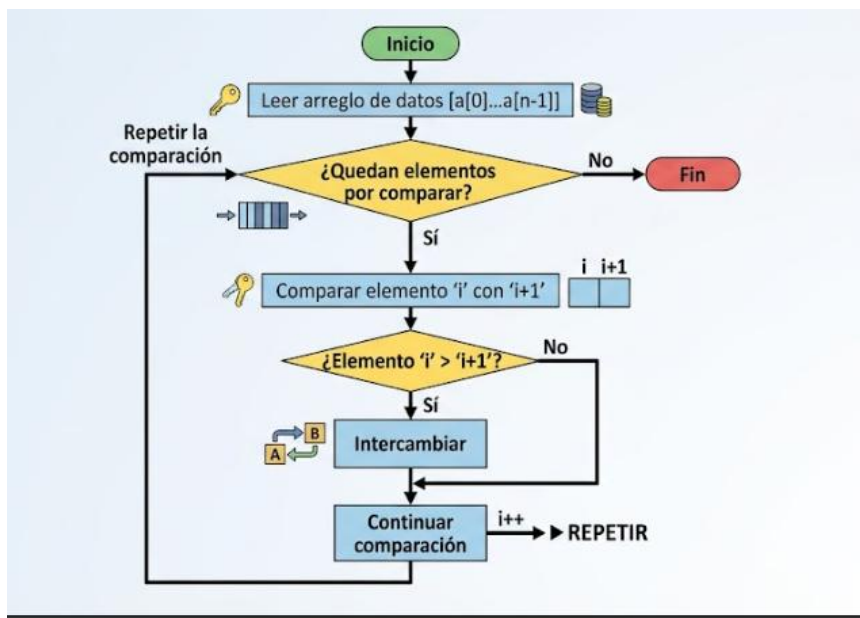
Cuarta pasada

- 3 y 1 → intercambio.

Resultado final:

1, 3, 5, 8, 9

Diagrama de Flujo:



Ventajas

- Es un algoritmo sencillo de comprender e implementar.
- Resulta adecuado para arreglos pequeños o como herramienta para el aprendizaje de algoritmos.

Desventajas

- Presenta un bajo rendimiento cuando el número de elementos es grande.
- Requiere realizar una gran cantidad de comparaciones e intercambios, lo que incrementa el tiempo de ejecución.

2.2 Ordenamiento por Selección (Selection Sort)

Descripción

El **ordenamiento por selección (Selection Sort)** es un algoritmo de ordenamiento que organiza una lista seleccionando, en cada recorrido, el elemento más pequeño (o el más grande, según el criterio de ordenamiento) de la parte no ordenada y colocándolo en la posición que le corresponde. A diferencia del algoritmo de burbuja, el ordenamiento por selección realiza un solo intercambio por cada pasada, lo que reduce la cantidad de movimientos de datos, aunque el número de comparaciones sigue siendo elevado.

Este algoritmo divide el arreglo en dos partes: una sección ya ordenada y otra que aún no ha sido ordenada. En cada iteración, busca el valor mínimo de la sección desordenada y lo intercambia con el primer elemento de dicha sección. El proceso continúa hasta que todos los elementos quedan organizados (Zohonero Martínez & Joyanes Aguilar, 2008).

Funcionamiento paso a paso

Supóngase el siguiente arreglo:

8, 5, 3, 9, 1

Primera pasada

- Se considera que el primer elemento (8) es el menor.
- Se compara con 5 → 5 es menor.
- Se compara con 3 → 3 es menor.
- Se compara con 9 → no cambia.
- Se compara con 1 → 1 es el menor.

Se intercambia el **8** con el **1**.

Resultado:

1, 5, 3, 9, 8

Segunda pasada

Ahora se busca el menor elemento desde la segunda posición.

- 5 es el menor.
- Se compara con 3 → 3 es menor.
- Se compara con 9 → no cambia.
- Se compara con 8 → no cambia.

Se intercambia **5** con **3**.

Resultado:

1, 3, 5, 9, 8

Tercera pasada

Se busca el menor entre:

5, 9, 8

El menor es **5**, por lo tanto no se realiza ningún intercambio.

Resultado:

1, 3, 5, 9, 8

Cuarta pasada

Se compara:

9 y 8

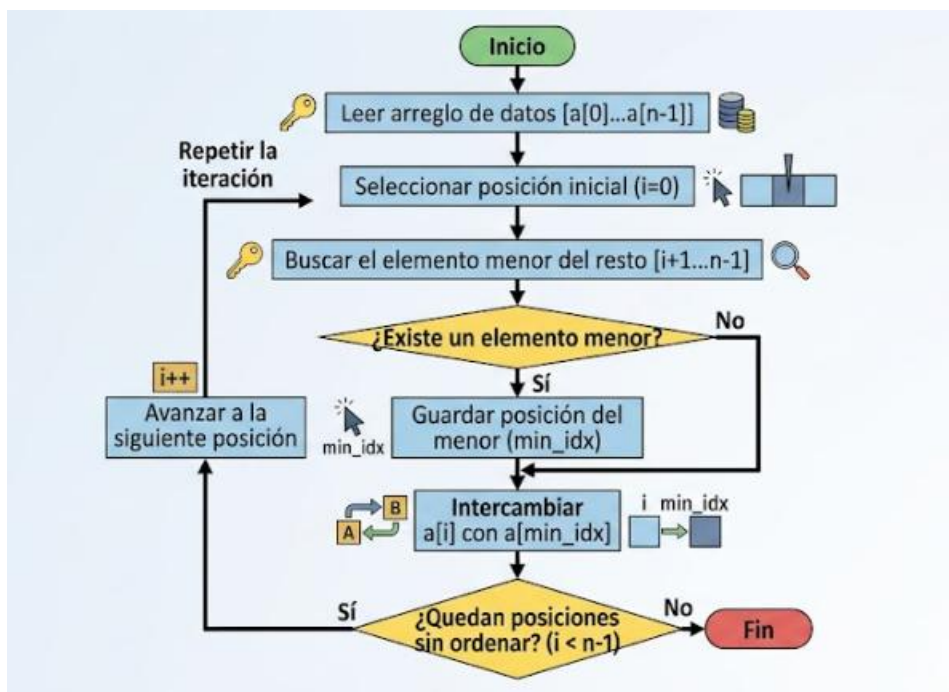
El menor es **8**.

Se intercambian.

Resultado final:

1, 3, 5, 8, 9

Diagrama de Flujo:



Ventajas

- Requiere pocos intercambios de datos, ya que solo realiza uno por cada recorrido del arreglo.
- Su implementación es sencilla, lo que facilita su comprensión y aplicación en programas de pequeño tamaño.

Desventajas

- Debe recorrer toda la parte desordenada del arreglo para encontrar el elemento mínimo en cada iteración, por lo que resulta poco eficiente con grandes cantidades de datos.
- Su complejidad temporal es $O(n^2)$ tanto en el mejor como en el peor caso, lo que lo hace considerablemente más lento que algoritmos más avanzados como Quick Sort.
-

2.3 Ordenamiento por Hundimiento (Sink Sort)

Descripción

El **ordenamiento por hundimiento (Sink Sort)** es una variante del algoritmo de **ordenamiento de burbuja (Bubble Sort)**. Su funcionamiento consiste en comparar elementos adyacentes del arreglo y realizar intercambios cuando se encuentran en un orden incorrecto. La diferencia principal radica en el sentido del recorrido: mientras Bubble Sort suele hacer que los elementos de mayor valor "suban" hacia el final del arreglo, Sink Sort permite que los elementos de menor valor "se hundan" hacia el inicio de la lista mediante comparaciones sucesivas.

En la literatura sobre algoritmos, el término **Sink Sort** no suele presentarse como un algoritmo independiente, sino como una variante o una forma de describir el comportamiento del Bubble Sort. Por ello, muchos libros de estructuras de datos explican únicamente Bubble Sort y consideran el hundimiento como parte de su funcionamiento (Fritelli et al., 2020; Zohonero Martínez & Joyanes Aguilar, 2008).

Funcionamiento paso a paso

Supóngase el siguiente arreglo:

8, 5, 3, 9, 1

El objetivo es que el elemento más pequeño vaya desplazándose hacia el inicio del arreglo.

Primera pasada (de derecha a izquierda)

- Se compara 9 con 1 → se intercambian.
- Se compara 3 con 1 → se intercambian.
- Se compara 5 con 1 → se intercambian.
- Se compara 8 con 1 → se intercambian.

Resultado:

1, 8, 5, 3, 9

El elemento 1 ya quedó en su posición definitiva.

Segunda pasada

Ahora se continúa con el resto del arreglo.

- Se compara 9 con 3 → se intercambian.
- Se compara 5 con 3 → se intercambian.
- Se compara 8 con 3 → se intercambian.

Resultado:

1, 3, 8, 5, 9

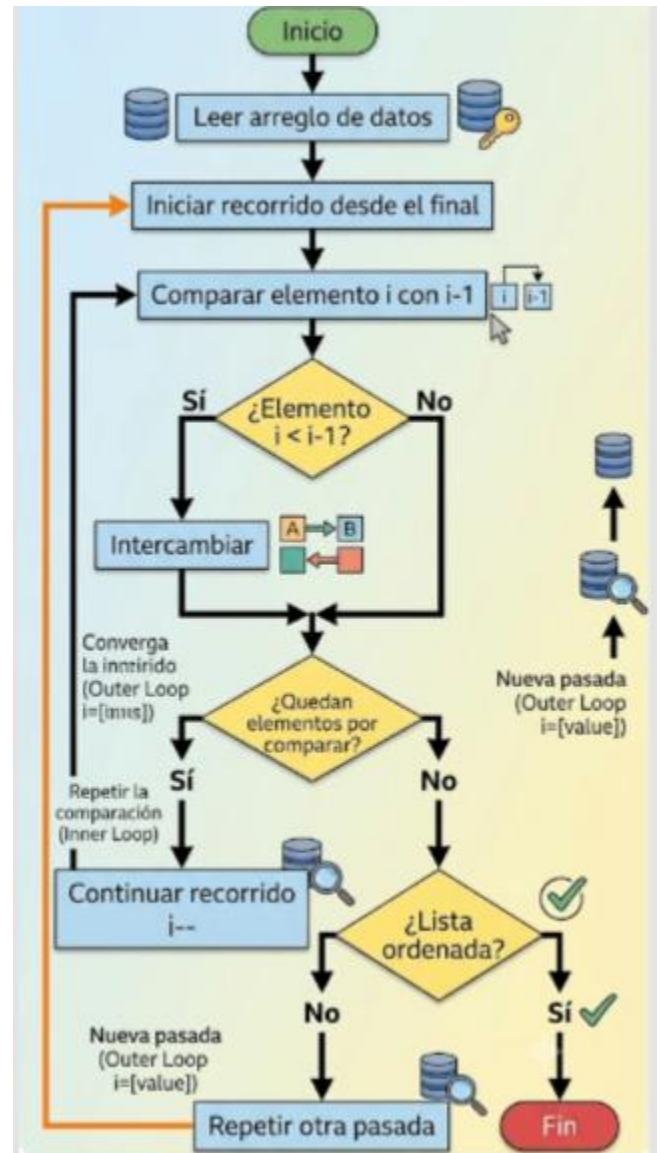
Tercera pasada

- Se compara 9 con 5 → correcto.
- Se compara 8 con 5 → se intercambian.

Resultado:

1, 3, 5, 8, 9

El arreglo ya se encuentra completamente ordenado.



Ventajas

- Es un algoritmo sencillo de comprender e implementar, especialmente para estudiantes que inician en el estudio de los algoritmos de ordenamiento.
- Comparte la estabilidad del Bubble Sort, ya que conserva el orden relativo de elementos con valores iguales cuando se implementa correctamente.

Desventajas

- Su rendimiento es bajo en arreglos de gran tamaño debido a la gran cantidad de comparaciones e intercambios que realiza.
- Presenta una complejidad temporal de $O(n^2)$, por lo que resulta menos eficiente que algoritmos como Quick Sort.

2.4 Ordenamiento Rápido (Quick Sort)

Descripción

El **ordenamiento rápido (Quick Sort)** es un algoritmo de ordenamiento basado en la estrategia de **divide y vencerás**. Su funcionamiento consiste en seleccionar un elemento del arreglo denominado **pivote** y reorganizar los demás elementos de manera que aquellos con valores menores queden a su izquierda y los mayores a su derecha. Una vez realizada esta partición, el mismo procedimiento se aplica de forma recursiva a cada una de las dos sublistas hasta que todo el arreglo queda ordenado.

Quick Sort es considerado uno de los algoritmos de ordenamiento más eficientes y utilizados debido a su excelente rendimiento promedio. Aunque en el peor de los casos su complejidad puede llegar a $O(n^2)$, normalmente alcanza una complejidad de $O(n \log n)$, lo que lo hace apropiado para ordenar grandes volúmenes de información (Zohonero Martínez & Joyanes Aguilar, 2008; Fritelli et al., 2020).

Funcionamiento paso a paso

Supóngase el siguiente arreglo:

8, 5, 3, 9, 1

Se elige como **pivote** el último elemento (**1**).

Primera partición

Se comparan todos los elementos con el pivote.

- $8 > 1$
- $5 > 1$
- $3 > 1$
- $9 > 1$

Como ninguno es menor que 1, el pivote se coloca al inicio del arreglo.

Resultado:

1 | 5, 3, 9, 8

El número **1** ya quedó en su posición definitiva.

Segunda partición

Ahora se trabaja únicamente con:

5, 3, 9, 8

Se toma como pivote el **8**.

Comparaciones:

- $5 < 8 \rightarrow$ queda a la izquierda.
- $3 < 8 \rightarrow$ queda a la izquierda.
- $9 > 8 \rightarrow$ queda a la derecha.

Resultado:

1 | 5, 3 | 8 | 9

El número **8** queda ordenado.

Tercera partición

Se ordena la sublista:

5, 3

Se toma como pivote el **3**.

Resultado:

1 | 3 | 5 | 8 | 9

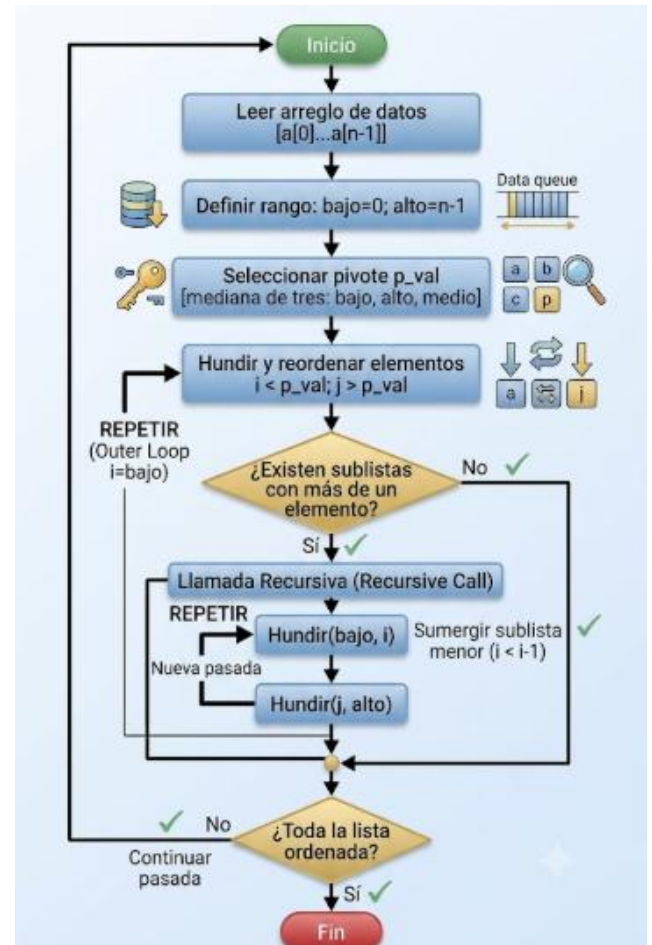
Todo el arreglo queda ordenado.

Ventajas

- Presenta un excelente rendimiento para ordenar grandes cantidades de datos, con una complejidad promedio de $O(n \log n)$.
- Es uno de los algoritmos de ordenamiento más utilizados en aplicaciones reales debido a su rapidez y eficiencia.

Desventajas

- En el peor de los casos, cuando el pivote no se elige adecuadamente, su complejidad puede aumentar hasta $O(n^2)$.
- Su implementación es más compleja que la de algoritmos básicos como Bubble Sort o Selection Sort, especialmente por el uso de recursividad.



4. Comparación entre los algoritmos de ordenamiento

4.1 Tabla comparativa de los algoritmos

Característica	Bubble Sort	Selection Sort	Sink Sort	Quick Sort
Método de ordenamiento	Compara elementos adyacentes e intercambia los que están en orden incorrecto.	Busca el elemento mínimo de la parte desordenada y lo coloca en su posición correcta.	Variante del Bubble Sort que desplaza los elementos menores hacia el inicio del arreglo.	Divide el arreglo utilizando un pivote y ordena recursivamente las particiones.
Cantidad de intercambios	Alta	Baja	Alta	Moderada
Facilidad de implementación	Muy fácil	Fácil	Fácil	Media-Alta
Uso recomendado	Arreglos pequeños o fines educativos.	Arreglos pequeños donde se desea reducir la cantidad de intercambios.	Aprendizaje de variantes de Bubble Sort y conjuntos pequeños de datos.	Grandes volúmenes de información y aplicaciones reales.

4.2 Diferencias clave en términos de eficiencia y facilidad de implementación

Los algoritmos de **Bubble Sort**, **Selection Sort** y **Sink Sort** pertenecen al grupo de algoritmos de ordenamiento básicos. Se caracterizan por ser sencillos de comprender e implementar, razón por la cual son ampliamente utilizados en el aprendizaje de la programación y las estructuras de datos. Sin embargo, presentan una eficiencia limitada debido a que, en promedio, requieren un número de operaciones proporcional al cuadrado de la cantidad de elementos ($O(n^2)$), lo que reduce su rendimiento cuando se trabaja con grandes volúmenes de información (Fritelli et al., 2020).

Por otro lado, **Quick Sort** emplea la estrategia de **divide y vencerás**, dividiendo el problema en subproblemas más pequeños mediante la selección de un pivote. Gracias a este enfoque, alcanza una complejidad promedio de $O(n \log n)$, ofreciendo un rendimiento considerablemente superior en comparación con los algoritmos básicos. No obstante, su implementación resulta más compleja debido al uso de recursividad y al proceso de partición del arreglo (Zohonero Martínez & Joyanes Aguilar, 2008).

En cuanto a la facilidad de implementación, **Bubble Sort** es generalmente considerado el algoritmo más sencillo, seguido por **Selection Sort** y **Sink Sort**, mientras que **Quick Sort** requiere

un mayor conocimiento de programación y estructuras de datos para implementarlo correctamente.

4.3 Escenarios donde cada algoritmo podría ser preferible

Bubble Sort

Puede ser la mejor opción cuando se trabaja con arreglos pequeños o cuando el propósito principal es comprender el funcionamiento de los algoritmos de ordenamiento. También resulta útil en aplicaciones educativas donde la simplicidad del algoritmo es más importante que su eficiencia.

Selection Sort

Es recomendable cuando el costo de realizar intercambios de datos es elevado, ya que realiza un único intercambio por cada recorrido del arreglo. Puede utilizarse en sistemas donde los movimientos de datos sean más costosos que las comparaciones.

Sink Sort

Puede emplearse en situaciones similares a Bubble Sort, especialmente con conjuntos pequeños de datos y fines didácticos. Su utilidad principal radica en mostrar una variante del proceso de ordenamiento mediante comparaciones entre elementos adyacentes.

Quick Sort

Es el algoritmo más adecuado para aplicaciones que manejan grandes cantidades de información, como sistemas de bases de datos, motores de búsqueda, aplicaciones financieras y programas de gestión empresarial. Su eficiencia promedio lo convierte en una de las opciones más utilizadas en el desarrollo de software moderno.

5. Reflexión crítica

5.1 Limitación común de los algoritmos estudiados

Una de las principales limitaciones que comparten los algoritmos de ordenamiento estudiados es que su rendimiento depende de las características del conjunto de datos y del tamaño de la información que se desea ordenar. En el caso de **Bubble Sort**, **Selection Sort** y **Sink Sort**, la complejidad temporal de $O(n^2)$ hace que su desempeño disminuya considerablemente cuando el número de elementos aumenta. Aunque Quick Sort ofrece una mejor eficiencia promedio, su rendimiento también puede verse afectado si la elección del pivote no es adecuada, llegando a presentar un comportamiento de $O(n^2)$ en el peor de los casos (Fritelli et al., 2020).

Otra limitación común es que ninguno de estos algoritmos resulta óptimo para todas las situaciones. La selección del algoritmo adecuado depende de factores como el tamaño del conjunto

de datos, la memoria disponible, la estabilidad requerida y el tiempo de ejecución esperado. Por ello, en el desarrollo de software es importante analizar las características del problema antes de elegir el método de ordenamiento más conveniente (Ayala San Martín, 2020).

5.2 Relevancia de estos algoritmos en la programación moderna

Aunque actualmente existen algoritmos de ordenamiento más eficientes, como **Merge Sort**, **Heap Sort** o **TimSort**, los algoritmos básicos continúan siendo fundamentales en la formación de los programadores. Su estudio permite comprender conceptos esenciales como las comparaciones, los intercambios de datos, la recursividad, la complejidad algorítmica y el análisis de eficiencia.

Además, estos algoritmos constituyen la base para entender técnicas de diseño de algoritmos más avanzadas y facilitan el aprendizaje de estructuras de datos y análisis de rendimiento. Por esta razón, continúan formando parte de los cursos universitarios de programación y estructuras de datos, ya que fortalecen el razonamiento lógico y la capacidad para resolver problemas computacionales (Zohonero Martínez & Joyanes Aguilar, 2008).

6. Conclusiones

El desarrollo de esta investigación permitió comprender que los algoritmos de ordenamiento desempeñan un papel fundamental en la organización y procesamiento eficiente de la información. Su utilización facilita operaciones como la búsqueda, el análisis de datos y la generación de reportes, mejorando el rendimiento general de numerosas aplicaciones informáticas.

En cuanto a los algoritmos estudiados, se identificó que **Bubble Sort** destaca por su simplicidad y facilidad de implementación, lo que lo convierte en una excelente herramienta para el aprendizaje. **Selection Sort** se caracteriza por reducir la cantidad de intercambios realizados durante el proceso de ordenamiento, aunque mantiene una complejidad temporal elevada. **Sink Sort**, considerado una variante de Bubble Sort, presenta un funcionamiento similar al desplazar los elementos menores hacia el inicio del arreglo mediante comparaciones sucesivas. Finalmente, **Quick Sort** sobresale por su eficiencia promedio de $O(n \log n)$, siendo una de las técnicas más utilizadas en aplicaciones que manejan grandes volúmenes de información.

Asimismo, se concluye que conocer estos algoritmos permite comprender conceptos esenciales de la programación, como el análisis de complejidad, la optimización de recursos y la selección de soluciones adecuadas según las características de cada problema. Aunque algunos de estos métodos han sido superados por algoritmos más avanzados, su estudio continúa siendo

indispensable en la formación de los programadores, ya que constituye la base para comprender técnicas más complejas y desarrollar software eficiente y de calidad (Fritelli et al., 2020; Zohonero Martínez & Joyanes Aguilar, 2008).

Referencias

Ayala San Martín, G. (2020). *Algoritmos y programación: mejores prácticas*. Fundación Universidad de las Américas Puebla (UDLAP).

Fritelli, V., Guzmán, A., & Tymoschuk, J. (2020). *Algoritmos y estructuras de datos* (2.^a ed.). Jorge Sarmiento Editor – Universitas.

Zohonero Martínez, I., & Joyanes Aguilar, L. (2008). *Estructuras de datos en Java*. McGraw-Hill España.